

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Васькин Виктор

Группа
J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Миграция ранее разработанного веб-приложения (PosiMax) на фреймворк Vue.js 3 с использованием Composition API. Целью работы является создание полноценного одностраничного приложения (SPA), настройка маршрутизации, централизованного управления состоянием через Pinia, использование Composables для переиспользования логики и организация работы с внешним REST API через библиотеку Axios

Ход работы

Процесс разработки включал следующие этапы:

- Инициализация проекта: Развернута структура приложения с поддержкой Vite, Vue Router и Pinia. Настроена интеграция с Bootstrap для стилизации
- Архитектура компонентов: Приложение переведено на компонентную модель. Созданы переиспользуемые компоненты (шапка Navbar.vue, карточка курса CourseCard.vue) и обертка страницы (BaseLayout.vue), что обеспечило чистоту кода и удобство поддержки
- Управление состоянием (Pinia): Настроено хранилище для авторизации пользователя. Внедрен плагин pinia-plugin-persistedstate для автоматической синхронизации состояния авторизации с localStorage (см. Рис. 1)
- Разработка Composables: Выделена общая логика управления темой оформления (светлая/темная) в отдельный файл useTheme.js, что позволило легко импортировать переключатель темы в любые компоненты (см. Рис. 2)
- Работа с API (Axios): Реализован слой взаимодействия с бэкендом через Axios. Настроен глобальный инстанс с базовым URL, к которому автоматически прикрепляется JWT-токен пользователя в заголовках запросов (см. Рис. 3)
- Навигация и Роутинг: Настроен vue-router с ленивой подгрузкой компонентов, что оптимизирует производительность SPA. Реализованы динамические маршруты для страниц курсов (параметр :id)
- Интерактивность: Добавлены методы для фильтрации курсов, работы с комментариями (через localStorage) и реализации плавных анимаций переходов страниц с помощью встроенного компонента <transition> (см. Рис. 4-6)

```

1  import { defineStore } from 'pinia'
2  import { ref } from 'vue'
3
4  export const useAuthStore = defineStore('auth', () => {
5    const user = ref(null)
6    const token = ref(null)
7
8    const login = (userData, accessToken) => {
9      user.value = userData
10     token.value = accessToken
11   }
12
13   const logout = () => {
14     user.value = null
15     token.value = null
16   }
17
18   return { user, token, login, logout }
19 }, {
20   persist: true
21 })

```

Рис. 1

```

1  import { ref } from 'vue'
2
3  export function useTheme() {
4    const theme = ref(localStorage.getItem('posimax-theme') || 'dark')
5
6    const applyTheme = (currentTheme) => {
7      document.body.setAttribute('data-theme', currentTheme)
8      document.documentElement.setAttribute('data-bs-theme', currentTheme)
9    }
10
11    const initTheme = () => {
12      applyTheme(theme.value)
13    }
14
15    const toggleTheme = () => {
16      theme.value = theme.value === 'dark' ? 'light' : 'dark'
17      localStorage.setItem('posimax-theme', theme.value)
18      applyTheme(theme.value)
19    }
20
21    return {
22      theme,
23      initTheme,
24      toggleTheme
25    }
26  }

```

Рис. 2

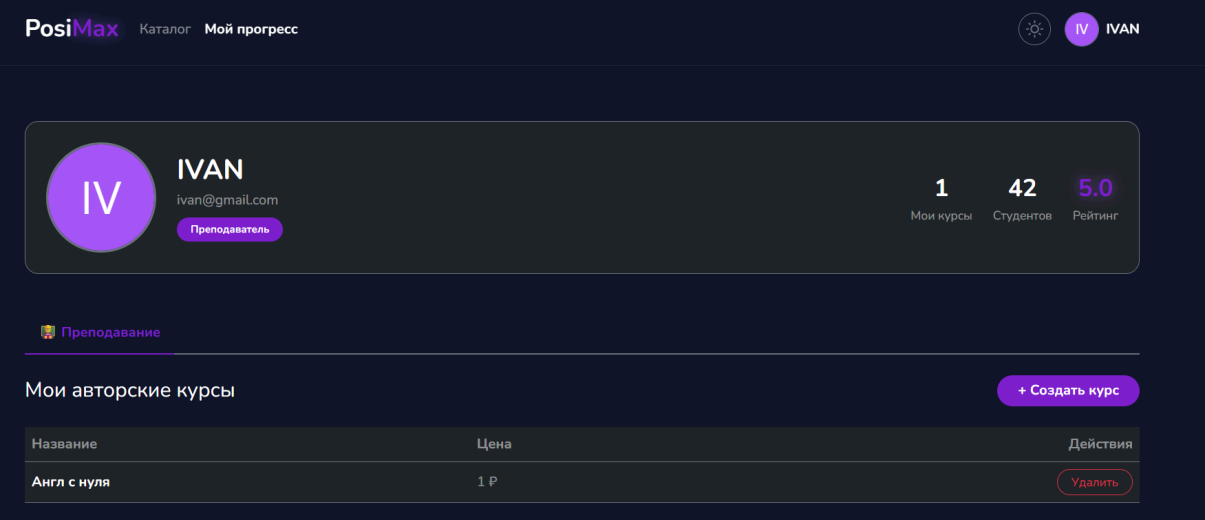


Рис. 5

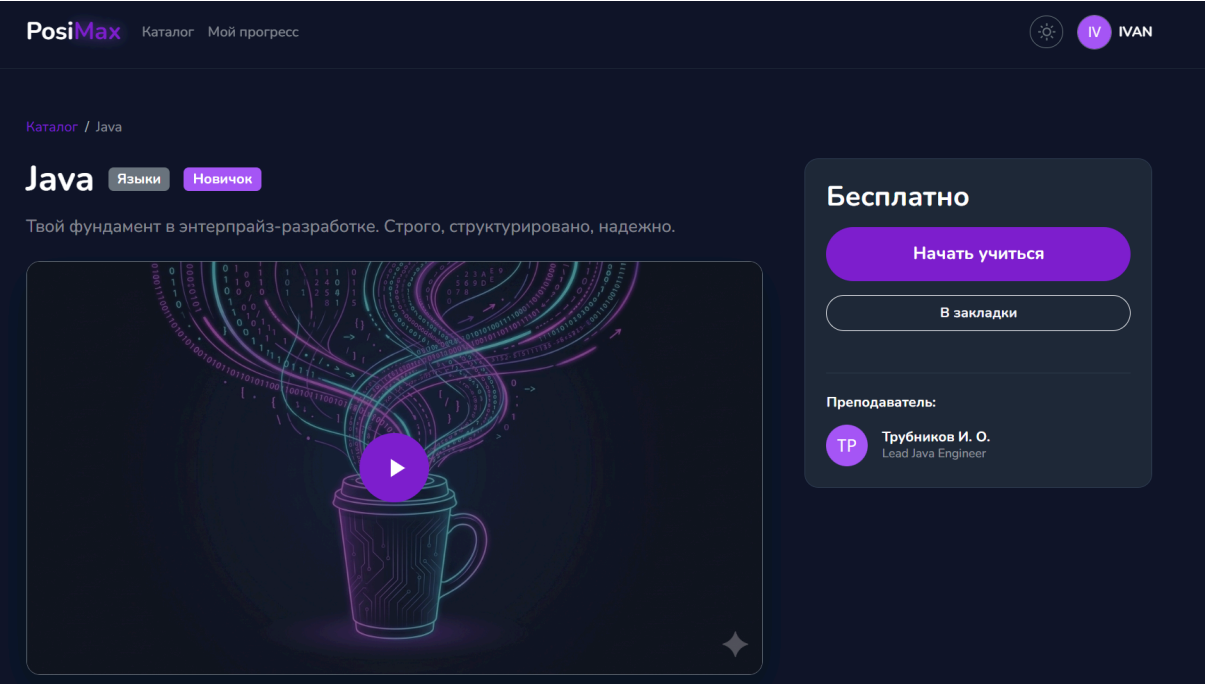


Рис. 6

Вывод

В ходе лабораторной работы я успешно мигрировал статическое приложение на современный фреймворк Vue.js 3. Я освоил архитектуру SPA, научился эффективно управлять состоянием приложения и делегировать логику в Composables. Проект стал значительно быстрее, адаптивнее и проще в расширении. Понимание принципов реактивности и компонентного подхода, полученное в этой работе, является ключевым навыком для разработки профессиональных веб-интерфейсов